EXP:6

11/08/26

Error Detection and Correction using Hamming code

AIM:

To write a program to implement error detection and error correction using the Hamming code.

ALGORITHM:

* Start the program
* Read the number of 4-bit data blocks (n) forming the input data stream.
* For each 4-bit data block $D = d_1\ d_2\ d_3\ d_4$, perform Hamming encoding:
  a) Arrange bits with parity bits at positions 1, 2 and 4 and data bits at position 3, 5, and 7.
  b) Compute the parity bits using even parity:
  $P_1 = d_1$ XOR d_2 XOR d_4 (covers bit positions 1, 3, 5, 7)
  $P_2 = d_1$ XOR d_3 XOR d_4 (covers bit positions 2, 3, 6, 7)
  $P_4 = d_2$ XOR d_3 XOR d_4 (covers bit positions 4, 5, 6, 7)
  c) Form the 7-bit Hamming code word: $P_1\ P_2\ d_1\ P_4\ d_2\ d_3\ d_4$.
* Simulate transmission by introducing a single-bit error at a chosen bit position (entered by the user) in the code.
* At the receiver, recompute the check bits from the received code word:
  $C_1 = P_1$ XOR d_1 XOR d_2 XOR d_4
  $C_2 = P_2$ XOR d_1 XOR d_3 XOR d_4

OUTPUT:

```
== HAMMING CODE - ERROR DETECTION AND CORRECTION

Enter number of 4-bit data blocks in the stream: 1

--Block 1--
Enter 4 data bits (space separated, eg: 1 0 1 1): 1 0 1 1
Original Data Bits : 1011
Encoded Hamming code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0=no error): 3
Transmitted code (with error induced at pos 3): 0100011

Received code : 0100011

Checking for errors....
    Error detected at bit position : 3 -> Correcting..
Corrected code : 0110011
Recovered Data Bits : 1011
```

formed from the check bits).

* If Error position = 0 : no error has occured.
* Else Extract the original 4 data bits from the corrected 7-bit code word.
* Repeat steps 3 to 8 for all n blocks in the data stream.
* Stop the program.

PROGRAM :-

```
def encode(d):
    """d: list of 4 bits [d1, d2, d3, d4] -> returns 7-bits
    hamming code list positions (1-indexed): 1=P1 2=P2 3=d1 4=P4
    5=d2 6=d3 7=d4"""
    h = [0]*7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
    h[0] = h[2]^h[4]^h[6]
    h[1] = h[2]^h[5]^h[6]
    h[3] = h[4]^h[5]^h[6]
    return h
def detect_and_correct(h):
    c1 = h[0]^h[2]^h[4]^h[6]
    c2 = h[1]^h[2]^h[5]^h[6]
    c4 = h[3]^h[4]^h[5]^h[6]
    error_pos = c1*1+c2*2+c4*4
    if error_pos != 0
        print(f"Error detected at bit position: {error_pos} -> correcting...")
```

-- Block 2 --

Enter 4 data bits (Space separated, eg. 1011) : 0 1 1 0

Original Data Bits : 0110

Encoded Hamming code (7 bits) : 1100110

Enter bit position to introduce error (1-7, 0=no error

Transmitted code (with error induced at pos 6) :

Received code : 1100100

Checking for errors...

Error detected at bit position : 6 → Correct

Corrected code : 1100100

Recovered Data Bits : 0110,

```
    else:
        print("No error detected.")
    return error_pos

def to_str(bits):
    return "".join(str(b) for b in bits)

def main():
    print("== HAMMING CODE - ERROR DETECTION AND CORRECTION ==\n")
    n = int(input("Enter number of 4-bits data blocks in the stream: "))
    for b in range(1, n+1):
        print(f"\n-- Block {b} --")
        d = list(map(int, input("Enter 4 data bit (space separated
                                eg. 1011): ").split()))

        h = encode(d)
        print(f"Original Data Bits : {to_str(d)}")
        print(f"Encoded Hamming code (7bits) : {to_str(h)}")
        pos = int(input("Enter bit position to introduce error
                                (1-7, 0=no error): "))

        if 1 <= pos <= 7:
            h[pos-1] ^= 1
            print(f"Transmitted code (with error induced):
        else:
            print(f"Transmitted code (no error induced:
                                {to_str(h)}")

        print(f"Received code : {to_str(h)}")
        print("Checking for errors...")
        detect_and_correct(h)
        print(f"Corrected code : {to_str(h)}")

        rd = [h[2], h[4], h[5], h[6]]
        print(f"Recovered Data Bits : {to_str(rd)}")

if __name__ == "__main__":
    main()
```

RESULT:

The program to implement error detection and correction using Hamming code concept was successfully